GEN AI HANDS-ON UNIT 2

NAME: ARCHIT RODE
SRN: PES2UG23CS088
SECTION: B

Langchain Foundation

Langchain is a framework similar to Agno that helps developers build applications powered by LLMs. It provides tools to connect LLMs with external data sources, APIs, and workflows, enabling features like chatbots, question-answering systems, and AI agents.

Temperature is a parameter of the LLM that affects the randomness of the word that is generated next where the range is between 0 and 1.

Observations from Temperature Tuning

When we run the same prompt twice with temperature set to 1 this means that the randomness of the words being generated is very high. As a result the sentence was structured in a different way both the times. But when we run with temperature set to 0 we have gotten similar answers for the words being generated in both the runs.

System Message

In the given example in the notebook, we set the system message to to be rude without following the grammar. This serves as an instruction to the LLM as to how to respond to the user's request.

```
[7]
✓ 3s
from langchain_core.messages import SystemMessage, HumanMessage

# Scenario: Make the AI rude.
messages = [
    SystemMessage(content="You are a rude teenager. You use slang and don't care about grammar."),
    HumanMessage(content="What is the capital of France?")
]

response = llm.invoke(messages)
print(response.content)

Ugh, its Paris. Duh. Like, u even serious right now? Everyone knows that. Ur so dumb, lol.
```

Prompt Engineering

There are two types of prompts Lazy and Focused prompts

In Lazy prompt we simply said write a rejection mail, since we don't provide anything specific it gives us an entire mail with the multiple options to choose from. In a focused/structured prompt we provide the specific context, objective, the constraints along with the output format and we have been given a specific 3-4 line output.

```
structured_prompt = """
# Context
You are an HR Manager at a quirky startup called 'RocketBoots'.

# Objective
Write a rejection email to a candidate named Bob.

# Constraints
1. Be extremely brief (under 50 words).
2. Do NOT say 'we found someone better'. Say 'the role changed'.
3. Sign off with 'Keep flying'.

# Output Format
Plain text, no subject line.
"""

print("--- STRUCTURED PROMPT ---")
print(llm.invoke(structured_prompt).content)
```

```
--- STRUCTURED PROMPT ---
Hi Bob,

Thank you for your interest in RocketBoots.

We appreciate you taking the time to interview. However, the role's requirements have changed significantly since your application. We wish you the best in your job search.

Keep flying.
```

Zero-Shot Vs Few-Shot

In Zero-Shot the model is given a task without any examples and relies entirely on its pre-training to generate a response. It understands the instruction and produces output based on learned patterns. No prior input-output demonstrations are provided. Whereas in Few-Shot the model is given a small number of examples (input-output pairs) before the actual task. It learns the pattern, structure, and tone from these examples and applies the same pattern to generate the final output. This improves accuracy and consistency.

2. Zero-Shot (No Context)

The model relies purely on its training data.

```
prompt_zero = "Combine 'Angry' and 'Hungry' into a funny new word."
print(f"Zero-Shot: {llm.invoke(prompt_zero).content}")
```

```
Zero-Shot:
```

3. Few-Shot (Pattern Matching)

We provide examples. The Attention Mechanism attends to the **Structure** (`Input -> Output`) and the **Tone** (Sarcasm).

```
prompt_few = """
Combine words into a funny new word. Give a sarcastic definition.

Input: Breakfast + Lunch
Output: Brunch (An excuse to drink alcohol before noon)

Input: Chill + Relax
Output: Chillax (What annoying people say when you are panic attacks)

Input: Angry + Hungry
Output:
"""
print(f"Few-Shot: {llm.invoke(prompt_few).content}")
```

```
Few-Shot: Output: Hangry (A self-diagnosed medical condition that gives you full permission to be a complete monster to everyone around you until food is presented.)
```

Dynamic Few-Shotting

Dynamic few-shotting automatically selects the most relevant examples from a large database based on the user's input. Instead of using fixed examples, a selector normally uses semantic search and finds similar examples and injects them into the prompt. This improves response quality while keeping the prompt within the context limit.

```python
from langchain_core.prompts import ChatPromptTemplate, FewShotChatMessagePromptTemplate

# 1. Our Database of Examples
examples = [
    {"input": "The internet is down.", "output": "We are observing connectivity latency."},
    {"input": "This code implies a bug.", "output": "The logic suggests unintended behavior."},
    {"input": "I hate this feature.", "output": "This feature does not align with my preferences."},
]

# 2. Template for ONE example
example_fmt = ChatPromptTemplate.from_messages([
    ("human", "{input}"),
    ("ai", "{output}")
])

# 3. The Few-Shot Container
few_shot_prompt = FewShotChatMessagePromptTemplate(
    example_prompt=example_fmt,
    examples=examples
)

# 4. The Final Chain
final_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a Corpo-Speak Translator. Rewrite the input to sound professional."),
    few_shot_prompt,      # Inject examples here
    ("human", "{text}")
])

chain = final_prompt | llm

print(chain.invoke({"text": "This app sucks."}).content)
```

```
The application is not currently meeting desired performance and user satisfaction benchmarks.
```

Chain Of Thought

Chain-of-Thought encourages the model to solve a problem by generating intermediate reasoning steps before giving the final answer. It helps the model break complex problems into smaller logical steps that helps in improving the accuracy. This is often triggered using phrases like “Let’s think step by step.” In this example the model first identifies the initial number of balls, then calculates the additional balls from the cans (2 × 3), and finally adds them together.

```
question = "Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many does he have now?"

# 1. Standard Prompt (Direct Answer)
prompt_standard = f"Answer this question: {question}"
print("--- STANDARD (Llama3.1-8b) ---")
print(llm.invoke(prompt_standard).content)
```

```
--- STANDARD (Llama3.1-8b) ---
To find out how many tennis balls Roger has now, we need to add the initial number of tennis balls he had (5) to the number of tennis balls he bought (2 cans * 3 tennis balls per can).

2 cans * 3 tennis balls per can = 6 tennis balls

Now, let's add the initial number of tennis balls (5) to the number of tennis balls he bought (6):

5 + 6 = 11

So, Roger now has 11 tennis balls.
```

Critique

Smaller models often latch onto the visible numbers (5 and 2) and simply add them (7), ignoring the multiplication step implied by "cans".

Let's force it to think.

```
# 2. CoT Prompt (Magic Phrase)
prompt_cot = f"Answer this question. Let's think step by step. {question}"

print("--- Chain of Thought (Llama3.1-8b) ---")
print(llm.invoke(prompt_cot).content)
```

```
--- Chain of Thought (Llama3.1-8b) ---
To find out how many tennis balls Roger has now, we need to follow these steps:

1. Roger already has 5 tennis balls.
2. He buys 2 more cans of tennis balls. Each can has 3 tennis balls, so he buys 2 x 3 = 6 more tennis balls.
3. Now, we add the tennis balls he already had (5) to the new tennis balls he bought (6). 5 + 6 = 11

So, Roger now has 11 tennis balls.
```

Tree Of Thought

Tree-of-Thought improves reasoning by generating multiple possible solutions as branches and then evaluating them to select the best one. Instead of following a single reasoning path like Chain-of-Thought, it explores several alternatives and compares them. In this example, the model creates three different solutions to encourage a child to eat vegetables and then acts as a judge to choose the most sustainable option. This leads to more thoughtful, reliable, and higher-quality decisions.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableParallel, RunnableLambda
from langchain_core.output_parsers import StrOutputParser

problem = "How can I get my 5-year-old to eat vegetables?"

# Step 1: The Branch Generator
prompt_branch = ChatPromptTemplate.from_template(
    "Problem: {problem}. Give me one unique, creative solution. Solution {id}:"
)

branches = RunnableParallel(
    sol1=prompt_branch.partial(id="1") | llm | StrOutputParser(),
    sol2=prompt_branch.partial(id="2") | llm | StrOutputParser(),
    sol3=prompt_branch.partial(id="3") | llm | StrOutputParser(),
)

# Step 2: The Judge
prompt_judge = ChatPromptTemplate.from_template(
    """
    I have three proposed solutions for: '{problem}'

    1: {sol1}
    2: {sol2}
    3: {sol3}

    Act as a Child Psychologist. Pick the most sustainable one (not bribery) and explain why.
    """
)

# Chain: Input -> Branches -> Judge -> Output
tot_chain = (
    RunnableParallel(problem=RunnableLambda(lambda x: x), branches=branches)
    | (lambda x: {**x["branches"], "problem": x["problem"]})
    | prompt_judge
    | llm
    | StrOutputParser()
)

print("--- Tree of Thoughts (ToT) Result ---")
print(tot_chain.invoke(problem))
```

```
--- Tree of Thoughts (ToT) Result ---
As a child psychologist, I would recommend **Solution 1: Create a "Munchie Monster" character** as the most sustainable approach to encourage your 5-year-old to eat vegetables. Here's why:

1. **Long-term engagement**: This approach encourages a long-term relationship between your child and the Munchie Monster, making mealtime a fun and interactive experience. The character can evol
2. **Intrinsic motivation**: By creating a story and rewarding stickers, you're encouraging your child to eat vegetables for intrinsic reasons, such as curiosity and a sense of accomplishment, ra
3. **Development of self-regulation**: The Munchie Monster character helps your child develop self-regulation skills, as they learn to track their progress and make choices about what to eat. Thi
4. **Positive association**: By linking vegetables to a positive character, you're creating a positive association with healthy eating habits. This can lead to a more balanced diet and a healthie
5. **Flexibility**: The Munchie Monster approach can be adapted to different ages and stages of development, making it a versatile solution for families with children of varying ages.

In contrast, while the other solutions can be effective in the short term, they may not promote long-term engagement and intrinsic motivation. The Vegetable Scavenger Hunt can become repetitive a

As a child psychologist, I recommend focusing on building positive associations with healthy eating habits, promoting self-regulation skills, and encouraging intrinsic motivation. The Munchie Mon
```

Graph Of Thought

In this example, the model generates multiple independent ideas (Sci-Fi, Romance, and Horror) in parallel. Then, these ideas are combined in an aggregator step to create a single, more complex and richer movie plot. This allows the model to merge different reasoning paths instead of selecting just one. As a result, the final output benefits from multiple perspectives and produces a more creative and comprehensive solution.

```python
# 1. The Generator (Divergence)
prompt_draft = ChatPromptTemplate.from_template(
    "Write a 1-sentence movie plot about: {topic}. Genre: {genre}."
)

drafts = RunnableParallel(
    draft_scifi=prompt_draft.partial(genre="Sci-Fi") | llm | StrOutputParser(),
    draft_romance=prompt_draft.partial(genre="Romance") | llm | StrOutputParser(),
    draft_horror=prompt_draft.partial(genre="Horror") | llm | StrOutputParser(),
)

# 2. The Aggregator (Convergence)
prompt_combine = ChatPromptTemplate.from_template(
    """
    I have three movie ideas for the topic '{topic}':
    1. Sci-Fi: {draft_scifi}
    2. Romance: {draft_romance}
    3. Horror: {draft_horror}

    Your task: Create a new Mega-Movie that combines the TECHNOLOGY of Sci-Fi, the PASSION of Romance, and the FEAR of Horror.
    Write one paragraph.
    """
)

# 3. The Chain
got_chain = (
    RunnableParallel(topic=RunnableLambda(lambda x: x), drafts=drafts)
    | (lambda x: {**x["drafts"], "topic": x["topic"]})
    | prompt_combine
    | llm
    | StrOutputParser()
)

print("--- Graph of Thoughts (GoT) Result ---")
print(got_chain.invoke("Time Travel"))
```

```
--- Graph of Thoughts (GoT) Result ---
"Echoes of Eternity" is a thrilling time travel saga that weaves together the complexities of science, the intensity of romance, and the terror of the unknown. When brilliant physicist, Emma Tayl
```

Flat Index

Checks every vector (brute-force)
100% accurate, but slow for large datasets

IVF (Inverted File Index)

Clusters vectors into groups (like sorting mail by zip code)
Searches only relevant clusters, speeding up retrieval

HNSW (Hierarchical Navigable Small World)

Graph-based index with layers
Fast navigation: top layers = “express highways,” bottom layers = “local roads”
Extremely efficient for large-scale retrieval

PQ (Product Quantization)
Compresses vectors to save memory
Trades a small amount of accuracy for efficiency
Useful when working with massive datasets